

AdotaPet - NP2 / S07

Aplicando DevOps na pratica

Guia passo-a-passo para o time DevOps dar continuidade ao trabalho

O que voces estao recebendo:

- Monorepo do AdotaPet (backend NestJS + frontend Next.js 16 + MySQL) rodando ponta a ponta via Docker Compose com **4 containers**.
- **Jenkinsfile** declarativo (esqueleto) + **Jenkins em container** (jenkins/Dockerfile customizada com Node 20 e Docker CLI).
- Script de **notificacao por e-mail** ja funcional, lendo NOTIFY_EMAIL do ambiente (sem hardcoded).
- Suite de testes com **cobertura >= 90%** (97% backend / 98% frontend) e threshold no Jest para o pipeline reprovar regressao.
- **MailHog** (4o container) capturando os e-mails para conferir na defesa pela Web UI.
- Este PDF com tudo que ja esta pronto e o passo-a-passo do que falta voces fazerem.

Sumario

1. Visao geral do que foi entregue
2. Como subir o projeto local (passo-a-passo)
3. Validacao end-to-end (Swagger, UI, DBeaver, MailHog)
4. O que ja esta pronto vs o que voces precisam fazer
5. Estrutura do projeto
6. Como rodar o Jenkins (em container)
7. Como preencher o Jenkinsfile (stages Install / Test / Build)
8. Como publicar a imagem no Docker Hub
9. Como preencher a secao 'Uso de IA' do README
10. Checklist de validacao final + mapeamento de criterios
11. FAQ e troubleshooting

1. Visao geral do que foi entregue

O monorepo do AdotaPet foi reorganizado em 3 etapas, cada uma em uma branch git, para deixar facil rastrear a evolucao:

- **main** - estado original do repositorio.
- **audit/fixes** - correcoes minimas para o projeto rodar (docker-compose com paths errados, READMEs desatualizados, workflows com filtro de path quebrado etc.).
- **feature/devops-ready** (esta) - tudo o que e **especifico da NP2 S07**: 4o container, Jenkins em container, Jenkinsfile skeleton, notify-email, cobertura escopada >=90%, README com secao Uso de IA.

Para o pessoal do Jenkins, a branch ativa eh **feature/devops-ready**. Voces podem partir dela, fazer merge na main quando estiver pronta, ou criar uma feature branch propria a partir dela.

Stack

- **Backend**: NestJS 11 + TypeScript 5.7 + Prisma 6 + MySQL 8 + JWT + Bcrypt + Swagger.
- **Frontend**: Next.js 16 (App Router) + React 19 + Tailwind 4 + RHF + Zod + Cypress.
- **Banco**: MySQL 8 em container, healthcheck via mysqladmin ping.
- **CI/CD**: Jenkins LTS em container, Node 20 + Docker CLI pre-instalados.
- **E-mail**: MailHog (SMTP fake + Web UI) para capturar notificacoes do pipeline.

2. Como subir o projeto local

Pre-requisitos

- Node.js 20+ (so necessario se quiser rodar fora do Docker).
- Docker Desktop (ou Docker Engine + Compose v2).
- Git.
- **DBeaver** (opcional, ja configurado para localhost:3307 - ver imagem original).

Passo 1 - clonar e configurar

```
git clone https://github.com/roger-inatel/Dev_ops.git
cd Dev_ops
```

```
# Copie o template de env e ajuste JWT_SECRET
cp .env.example .env          # Linux/macOS
Copy-Item .env.example .env   # Windows PowerShell
```

*O arquivo **.env.example** ja vem com defaults sensatos. Voces SO precisam trocar o **JWT_SECRET** antes da entrega final (gera com openssl rand -hex 32).*

Passo 2 - subir os 4 containers

```
docker compose up --build -d
```

```
# Conferir que subiu:
docker compose ps
```

```
# Voces devem ver 4 containers:
# adotapet_mysql      (healthy, porta 3307)
# adotapet_backend    (porta 3010)
# adotapet_frontend   (porta 3011)
# adotapet_mailhog     (portas 1025 + 8025)
```

IMPORTANTE - portas mudaram em relacao aos defaults para evitar conflito com outros projetos:

Servico	Porta no HOST	Porta no container	URL / Acesso
MySQL	3307	3306	DBeaver: localhost:3307 / root / root
Backend (NestJS)	3010	3000	http://localhost:3010/docs (Swagger)
Frontend (Next.js)	3011	3000	http://localhost:3011
MailHog SMTP	1025	1025	(usado pelo notify-email.js)
MailHog Web UI	8025	8025	http://localhost:8025

Para mudar uma porta, edite .env e suba de novo: docker compose up -d (nao precisa rebuild).

3. Validacao end-to-end

Sequencia rapida para provar que tudo esta funcional. Cada comando deve responder com o resultado descrito - se algum nao bater, eh porque algo nao subiu (cheque docker compose logs).

a) Backend Swagger e health

```
curl -s -o NUL -w "HTTP %{http_code}\n" http://localhost:3010/docs
# esperado: HTTP 200

curl http://localhost:3010/
# esperado: Hello World!
```

b) Banco - via DBeaver ou CLI

Use a conexao DBeaver que ja existe (host: localhost, porta: 3307, user: root, senha: root, db: adotapet). Voce deve ver as tabelas criadas pelas migrations do Prisma (User, Pet, Organization, AdoptionRequest, ResponsibilityTerm, etc).

```
# Ou via CLI dentro do container:
docker exec adotapet_mysql mysql -uroot -proot adotapet -e "SHOW TABLES;"
```

c) Cadastro + login (API)

```
# 1) Criar um usuario ADOPTER
curl -X POST http://localhost:3010/users \
  -H "Content-Type: application/json" \
  -d '{"fullName":"Joao","email":"joao@test.com","password":"Str0ng@Pass1","role":"ADOPTER"}'

# 2) Login - deve devolver { access_token, user }
curl -X POST http://localhost:3010/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"joao@test.com","password":"Str0ng@Pass1"}'
```

d) Frontend (UI)

- <http://localhost:3011> - home, lista de pets.
- <http://localhost:3011/login> - login, usa o usuario criado acima.
- <http://localhost:3011/dashboard> - sem token, redireciona para /login (protecao via src/proxy.ts).

Atencao Next.js 16: o que antes era 'middleware.ts' agora se chama 'proxy.ts'. Eh deliberado (breaking change da v16). Nao recriem um middleware.ts.

e) Notificacao por e-mail + MailHog

```
cd scripts
npm install # instala nodemailer (so na primeira vez)

# Windows PowerShell:
$env:NOTIFY_EMAIL='time@adotapet.local'
$env:SMTP_HOST='localhost'; $env:SMTP_PORT='1025'
$env:BUILD_STATUS='SUCCESS'; $env:BUILD_NUMBER='1'
$env:JOB_NAME='manual-test'
node notify-email.js

# Linux/macOS:
NOTIFY_EMAIL=time@adotapet.local SMTP_HOST=localhost SMTP_PORT=1025 \
  BUILD_STATUS=SUCCESS BUILD_NUMBER=1 JOB_NAME=manual-test \
  node notify-email.js

# Abrir o inbox e ver o e-mail capturado:
```

```
# http://localhost:8025
```

Se o script falhar com 'NOTIFY_EMAIL ausente' e código de saída 1, está correto - foi feito de propósito para o pipeline NÃO silenciar.

4. O que ja esta pronto vs o que voces fazem

Os criterios eliminatórios do PDF do professor são claros: **(a)** não usar GUI do Jenkins para configurar stages, **(b)** não copiar IA sem entender, **(c)** todos os membros tem que ter commits relevantes. Por isso a maior parte do **codigo de stages do Jenkinsfile** ficou para voces escreverem.

Item	Status	Detalhe
Aplicacao rodando ponta a ponta	PRONTO	4 containers sobem com 'docker compose up --build'. Validado neste guia.
Cobertura de testes >= 90%	PRONTO	Threshold do Jest forca o pipeline a falhar se cair. 67+22 testes verdes.
4o container + comunicacao + volumes	PRONTO	mysql + backend + frontend + mailhog; backend<->mysql, back<->mailhog.
Jenkins em container	PRONTO	jenkins/Dockerfile + docker-compose.jenkins.yml. Node 20 + Docker CLI.
Script de e-mail com env var	PRONTO	scripts/notify-email.js le NOTIFY_EMAIL; testado contra MailHog.
Jenkinsfile com estrutura das stages	PARCIAL	Skeleton com Checkout/Notify/post prontos. TODOs nas demais.
Imagem no Docker Hub	TIME FAZ	TIME FAZ: criar conta, repo, credencial 'dockerhub' no Jenkins.
Conteudo dos sh dentro do Install/Test/Build	TIME FAZ	TIME FAZ: cada integrante escreve uma stage (commits relevantes).
Secao 'Uso de IA' do README	TIME FAZ	TIME FAZ: modelos, prompts, dinamica, partes feitas a mao.
Defesa Q&A; com dominio do projeto	TIME FAZ	TIME FAZ: revisar este PDF + Jenkinsfile + docker-compose juntos.

5. Estrutura do projeto

Dev_ops/	
README.md	-> ponto de entrada (NP2 + Uso de IA)
.env.example	-> template das variaveis (portas, segredos)
.gitignore	-> ignora node_modules, .env, coverage, dist
Jenkinsfile	-> pipeline declarativo (TIME preenche TODOs)
docker-compose.yml	-> 4 containers (app stack)
docker-compose.jenkins.yml	-> Jenkins em container separado
backend/	-> NestJS + Prisma
src/	-> codigo (controllers, services, guards, ...)
prisma/	-> schema + 3 migrations
Dockerfile	-> imagem da API (Node 22 + nest build)
.env.example	-> env para rodar fora do Docker
package.json	-> scripts: build/lint/test/test:cov... Jest com coverageThreshold 90%
frontend/	-> Next.js 16
src/	-> App Router + components + contexts
proxy.ts (em src/)	-> protecao de rotas (era 'middleware.ts')
Dockerfile	-> imagem da UI (Node 22 + next build)
jest.config.ts	-> Jest com coverageThreshold 90%
scripts/	-> ESPECIFICO da NP2
notify-email.js	-> script chamado pela stage Notify
package.json	-> dependencia: nodemailer
README.md	-> uso do script
jenkins/	-> ESPECIFICO da NP2
Dockerfile	-> Jenkins LTS + Node 20 + Docker CLI + plugins
docs/	-> documentacao
matriz-permissoes.md	-> acesso por rota (back)
fluxo-adocao.md	-> estados + diagrama
np2/	-> este guia

6. Como rodar o Jenkins (em container)

O PDF do professor exige Jenkins em container - e não 'instalado direto na máquina'. Por isso entregamos uma imagem customizada (jenkins/Dockerfile) que já vem com tudo que o pipeline precisa: Node 20, Docker CLI, e os plugins (workflow, git, junit, htmlpublisher, docker-workflow, email-ext).

Passo 1 - subir o container

```
# A partir da raiz do repo:
docker compose -f docker-compose.jenkins.yml up -d --build

# Pegar a senha inicial do admin:
docker exec adotapet_jenkins cat /var/jenkins_home/secrets/initialAdminPassword

# Abrir http://localhost:8080 e fazer o setup inicial:
#   - pode pular plugins recomendados (a imagem já tem os necessários);
#   - criar um usuário admin.
```

Passo 2 - criar o Job apontando para o Jenkinsfile

- **New Item** -> nome adotapet-pipeline -> tipo **Pipeline**.
- Na configuração do job: seção **Pipeline** -> **Definition**: *Pipeline script from SCM* (este passo é permitido pelo PDF).
- **SCM**: Git. URL: o repositório (https://github.com/roger-inatel/Dev_ops.git). Branch: **/feature/devops-ready* (ou main após o merge).
- **Script Path**: Jenkinsfile.
- Salvar e voltar para a tela do job.

Passo 3 - configurar NOTIFY_EMAIL

Manage Jenkins -> **Configure System** -> **Global properties** -> marcar *Environment variables* -> **Add**: name=NOTIFY_EMAIL, value=(email do destinatário).

NAO coloquem o e-mail no Jenkinsfile nem no docker-compose. Esse é um critério explícito do PDF.

Passo 4 - como o Jenkins fala com o MailHog

O container do Jenkins (compose próprio) NAO está na mesma rede do compose principal. Por isso, o Jenkinsfile aponta para SMTP_HOST=host.docker.internal (default) - o Docker Desktop resolve isso para o IP do host, alcançando o MailHog na porta 1025 do host. Em Linux, o compose do Jenkins já tem extra_hosts: host.docker.internal:host-gateway para o mesmo efeito funcionar.

7. Como preencher o Jenkinsfile

Abram o arquivo Jenkinsfile na raiz do repo. Vocas vao encontrar 5 stages declaradas - 3 delas com // TODO time. Substituam cada TODO por comandos reais. **Dica:** para 'commits relevantes de todos', dividam uma stage por integrante.

Stage 'Install' - sugestao

```
stage('Install') {
  steps {
    sh 'cd backend && npm ci'
    sh 'cd frontend && npm ci'
    sh 'cd scripts && npm ci'
  }
}
```

Stage 'Test' - sugestao

```
stage('Test') {
  steps {
    sh 'cd backend && npx prisma generate'
    sh 'cd backend && npm run test:cov'
    sh 'cd frontend && npm test -- --coverage --ci'
    // 0 coverageThreshold 90% no Jest faz o build quebrar se cair.
  }
}
```

Stage 'Build' - sugestao

```
stage('Build') {
  steps {
    sh 'cd backend && npm run build'
    sh 'docker build -t SEU_DOCKER_USER/adotapet-backend:${BUILD_NUMBER} \
      -f backend/Dockerfile backend'
    withCredentials([usernamePassword(
      credentialsId: 'dockerhub',
      usernameVariable: 'DH_USER',
      passwordVariable: 'DH_PASS')]) {
      sh 'echo $DH_PASS | docker login -u $DH_USER --password-stdin'
      sh 'docker push SEU_DOCKER_USER/adotapet-backend:${BUILD_NUMBER}'
      sh 'docker tag SEU_DOCKER_USER/adotapet-backend:${BUILD_NUMBER} \
        SEU_DOCKER_USER/adotapet-backend:latest'
      sh 'docker push SEU_DOCKER_USER/adotapet-backend:latest'
    }
  }
}
```

Atencao: antes de descomentar o 'docker push', criem a credencial dockerhub em Manage Jenkins -> Credentials -> System -> Global -> Add Credentials (tipo: Username with password, ID exato: dockerhub).

Stages 'Checkout' e 'Notify' (ja prontas)

Nao precisam mexer. 'Checkout' faz o clone do SCM. 'Notify' chama scripts/notify-email.js que ja le NOTIFY_EMAIL do ambiente. O bloco post { always {} } arquiva pacote + relatorio + junit (critério 4 do PDF). O bloco post { failure {} } reenvia notificacao com BUILD_STATUS=FAILURE.

8. Publicar a imagem no Docker Hub

- Criem conta em <https://hub.docker.com/> (usem o usuario do time).
- Criem um **repositorio publico**, ex: SEU_USER/adotapet-backend.
- No Jenkins, criem a credencial conforme passo anterior.
- Rodem o pipeline. Apos sucesso, voltem em https://hub.docker.com/r/SEU_USER/adotapet-backend.
- Copiem o link e adicionem ao README na secao 'Publicacao' ou no proprio topo. Tambem entreguem este link na tarefa do Teams (critério explicito).

Para testar o que esta publicado, peguem a tag desejada e rodem: `docker run --rm -p 3010:3000 SEU_USER/adotapet-backend:latest`. Lembrem: esse run precisa de um MySQL acessivel - use a rede do compose principal ou suba so o mysql antes.

9. Como preencher a secao 'Uso de IA' do README

Esta secao eh **obrigatoria** pelo PDF do professor (secao 5) e **sejam honestos**. A avaliacao nao pune o uso de IA - pune a falta de transparencia. Copiar IA sem entender eh criterio eliminatorio. Abram README.md na raiz e encontrem a secao *Uso de IA*; ela esta com placeholders, basta preencher:

Modelos utilizados

Listem os modelos usados pelo time. Ex: Claude (Sonnet 4.5), ChatGPT (GPT-4), Cursor, Copilot, Gemini. Se algum nao foi usado, removam.

Para que foram usados

Sejam especificos. Exemplos honestos: 'geracao do esqueleto do Jenkinsfile', 'debugging do docker-compose com erro de rewrite no Next', 'gerar este guia em PDF', 'revisar matriz de permissoes', 'sugerir specs de Jest'.

Exemplos de prompts (pelo menos 3)

Coloquem o prompt **real** que voces usaram (copy-paste). Para cada, registrem: aceita / ajustada / descartada e o porque. Exemplos:

- 1) Prompt: "Como adicionar coverageThreshold no Jest do NestJS com jest-junit como reporter pro Jenkins ler?"
Resposta: aceita; ajustamos a porcentagem de branches para 75.
- 2) Prompt: "O rewrite do Next nao funciona dentro do Docker, ele chama localhost:3000 em vez do container backend. Por que?"
Resposta: ajustada; a IA sugeriu mudar para route handler, nos escolhemos passar BACKEND_INTERNAL_URL como ARG no Dockerfile.
- 3) Prompt: "Sugira um Jenkinsfile declarativo para stages Install / Test / Build / Notify, sem preencher os comandos (so esqueleto)."
Resposta: aceita o esqueleto; cada um do time escreveu os sh do proprio stage para garantir entendimento.

Dinamica de uso

Como o time usou IA? Exemplos: 'cada PR foi revisado pelo Claude antes do merge', 'usamos ChatGPT em pair programming para tirar duvidas de Prisma', 'usamos Copilot inline para testes Jest', etc.

O que NAO foi feito por IA

Destaquem o que cada integrante fez a mao. Ex: 'definicao das regras de negocio da adocao', 'desenho da matriz de permissoes', 'criacao da credencial Docker Hub e do Job no Jenkins'.

10. Checklist final + mapeamento dos criterios

Tabela completa cruzando o que o PDF do professor avalia com o que ja esta atendido vs o que cabe ao time. Levem este PDF impresso na defesa.

#	Critério do PDF	Onde esta atendido
1	Aplicacao funcional, testes >= 90%	Back: 97% stmts (67 tests); Front: 98% stmts (22 tests). Threshold no Jest.
2	Dockerfile (Jenkins em container, reproduzível)	jenkins/Dockerfile (Node 20 + Docker CLI + plugins) + backend/frontend Dockerfile.
3	Jenkinsfile - Testes / Build / Notificacao	Jenkinsfile na raiz (skeleton; Notify e post.always prontos).
4	Artefatos no Jenkins (pacote + relatorio)	post { always { archiveArtifacts ... junit ... } } no Jenkinsfile.
5	E-mail nao hardcoded - via env var	scripts/notify-email.js le NOTIFY_EMAIL; exit 1 se faltar.
6	Imagem publicada no Docker Hub	TIME FAZ: stage Build comentado no Jenkinsfile; precisa de credencial.
7	Docker Compose 4+ containers (mix local + docker)	docker-compose.yml: mysql + backend + frontend + mailhog.
8	Comunicacao entre containers	backend<->mysql (Prisma), backend/Jenkins<->mailhog (SMTP).
9	Volumes para persistencia	mysql_data, backend_uploads, jenkins_home.
10	Commits relevantes de todos os membros	TIME FAZ: cada integrante deve commitar pelo menos uma stage.
11	README completo + secao 'Uso de IA'	README.md tem secao 'Uso de IA' como template - TIME PREENCHE.
12	Uso transparente de IA	TIME FAZ: registrar modelos, prompts, dinamica no README.
13	Defesa Q&A - todos sabem do projeto	TIME FAZ: revisar este guia em grupo antes da defesa.
14	DevOps geral (coerencia)	Auditoria + reorganizacao pre-rodadas; este guia documenta as decisoes.

Checklist 5 minutos antes da defesa

- Stack do projeto sobe? docker compose up -d -> 4 containers verdes?
- Swagger abre em <http://localhost:3010/docs>?
- Frontend abre em <http://localhost:3011>?
- DB Beaver conecta em localhost:3307?
- Jenkins abre em <http://localhost:8080>?
- Job 'adotapet-pipeline' configurado pelo SCM (NAO pela GUI)?
- Variavel NOTIFY_EMAIL setada em Configure System?
- Credencial 'dockerhub' criada (se ja publicaram)?
- Pipeline roda 'Build Now' sem erro?
- MailHog em <http://localhost:8025> mostra o e-mail apos o build?
- Imagem aparece em hub.docker.com/r/SEU_USER/adotapet-backend?
- Secao 'Uso de IA' do README preenchida?
- Todos os membros tem commits relevantes no historico?

11. FAQ e troubleshooting

Port already allocated ao subir o compose

Algum outro projeto seu esta usando a porta 3010, 3011, 3307, 1025 ou 8025. Edite .env e mude o numero (variaveis BACKEND_HOST_PORT, FRONTEND_HOST_PORT, etc.). Depois docker compose up -d.

Erro de prisma generate dentro do container backend

O Dockerfile do backend ja roda npx prisma generate. Se voce rodar fora do Docker e esquecer, vai dar 'UserRole has no exported member'. Solucao: cd backend && npx prisma generate.

Frontend mostra 404 em /api-backend/*

O rewrite do Next eh compilado em **build time**. Se voce mexer em next.config.ts ou mudar BACKEND_INTERNAL_URL, eh preciso **rebuild**: docker compose up --build -d frontend.

Notify-email diz 'NOTIFY_EMAIL ausente'

Eh proposital - o script falha quando voce esquece de definir a variavel. No Jenkins, defina em *Configure System > Global properties*. Em teste manual, exporte antes de chamar o script.

Jenkins nao acha 'docker' no PATH

Se voce buildou a imagem do Jenkins sem o nosso jenkins/Dockerfile, vai faltar Docker CLI. Use sempre docker compose -f docker-compose.jenkins.yml up --build a partir da raiz do repo.

Login pelo frontend nao popula o menu

Antes desta versao, o backend devolvia apenas o token. Foi corrigido para devolver tambem o objeto user. Se persistir, conferir backend/src/modules/auth/auth.service.ts e o spec.

MySQL nao sobe (porta 3306 / 3307 ocupada)

Em geral acontece quando ha um MySQL nativo no Windows. Pare o servico ou mude a porta no .env (MYSQL_HOST_PORT=3308, por ex.). Lembre-se de atualizar a conexao do DBeaver tambem.

Sucesso na defesa. Quando duvidarem entre 'usar GUI' e 'colocar no Jenkinsfile', a resposta eh sempre Jenkinsfile.